VINUNIVERSITY

Data Foundations

AICB-P1 · Ngày 7 · Embedding & Vector Store

Tên Giảng Viên

VinUniversity · Phase 1 · Tuần 1 · 2026

HÃY SUY NGHĨ...

“Agent trả lời sai vì model yếu, hay vì nó không có đúng dữ liệu để suy luận?”

Giữ câu hỏi này trong đầu khi học bài hôm nay

Nội Dung Bài Học

1. Data strategy cho AI product
2. Agent memory architecture
3. Embeddings
4. Vector store
5. Kết nối agent với data
6. Chunking & retrieval basics
7. Lab 7 + deliverable
8. Preview Day 08

Mục Tiêu Ngày 7

- Phân biệt được **knowledge data, operational data, contextual data**
- Hiểu **embedding** là lớp biểu diễn nghĩa, không chỉ là một API call
- Hiểu **vector store** lưu gì, tìm gì, và lọc bằng metadata ra sao
- Mô tả được pipeline **Document** → **Chunk** → **Embed** → **Store** → **Query** → **Inject**
- Build được một **mini retrieval integration** để nối agent với dữ liệu riêng

Deliverable Cuối Ngày

> Data inventory + chunking / embedding script + vector store index + semantic search demo + retrieval-enabled answer function

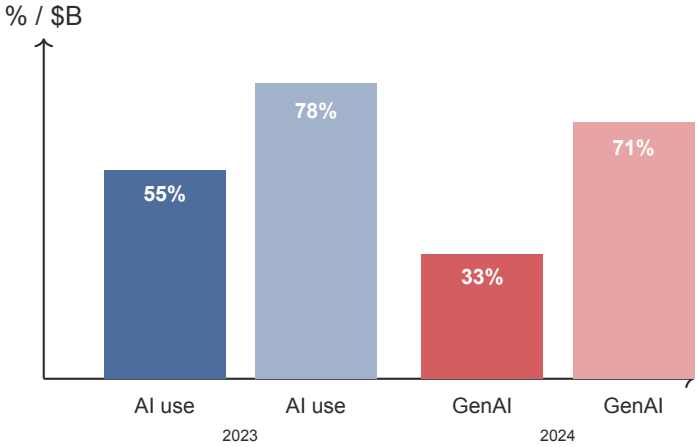
- 1 bộ dữ liệu mẫu đã được chunk và index
- 1 script truy vấn semantic search có trả kết quả liên quan
- 1 hàm trả lời sử dụng context retrieve được thay vì hỏi LLM “chay”

01

Data Strategy Cho Sản Phẩm AI

Không phải dữ liệu nào cũng nên đưa vào agent; chọn đúng dữ liệu quan trọng hơn nạp thật nhiều

Vì Sao Day 07 Quan Trọng Với AI Product?



- AI adoption đã tăng rất nhanh.
- Khi tổ chức dùng AI nhiều hơn, câu hỏi chuyển từ “dùng model nào” sang “agent được phép biết gì”.
- Nếu dữ liệu sai, cũ, bẩn, hoặc không truy được đúng lúc, output sẽ yếu dù model mạnh.

Based on Stanford AI Index 2025 economy chapter.

Garbage In, Garbage Out — Tại Sao Data Quyết Định Output

VINUNIVERSITY

Dữ liệu bẩn / thiếu

- PDF scan lỗi OCR
- Policy cũ 2 năm chưa cập nhật
- Chunk cắt giữa câu
- Không có metadata

Kết quả: Agent hallucinate, trả lời sai, user mất niềm tin.

Dữ liệu sạch / đầy đủ

- Text đã chuẩn hóa, metadata đầy đủ
- Nguồn rõ ràng, có version
- Chunk theo section hợp lý
- Filter theo category + freshness

Kết quả: Retrieve đúng, answer grounded, có trích nguồn.

Lưu ý: Đổi model đắt hơn không sửa được data bẩn. Hãy sửa data trước.

3 Loại Data Agent Cần

VINUNIVERSITY

Knowledge Data

Tài liệu, policy, SOP, FAQ, manual, hợp đồng, bài viết nội bộ.

Operational Data

Database, trạng thái đơn hàng, ticket, CRM records, logs, transactions.

Contextual Data

Session history, user profile, preferences, recent actions, channel context.

Knowledge data phù hợp với retrieval; **operational data** thường cần query có kiểm soát; **contextual data** nên được inject ngắn và đúng lúc.

Knowledge Data — Chi Tiết

VINUNIVERSITY

Đặc điểm

- Ít thay đổi (static hoặc versioned)
- Dạng text dài: tài liệu, hướng dẫn, hợp đồng
- Cần chunk + embed để agent truy xuất
- Nguồn sự thật chính cho retrieval

Ví dụ thực tế

- FAQ / SOP nội bộ công ty
- Chính sách bảo hành, đổi trả
- Tài liệu kỹ thuật sản phẩm
- Legal contracts, compliance docs

Retrieval fit: Rất cao — đây là kiểu data lý tưởng cho vector store.

Operational Data — Chi Tiết

VINUNIVERSITY

Đặc điểm

- Thay đổi liên tục (real-time hoặc near real-time)
- Dạng structured: SQL rows, JSON, logs
- Thường truy bằng API / SQL, không embed
- Cần access control chặt chẽ

Ví dụ thực tế

- Trạng thái đơn hàng, tracking
- CRM records, ticket history
- Server logs, transaction records
- Inventory levels, pricing

Retrieval fit: Thấp — dùng function calling / SQL query thay vì semantic search.

Contextual Data — Chi Tiết

VINUNIVERSITY

Đặc điểm

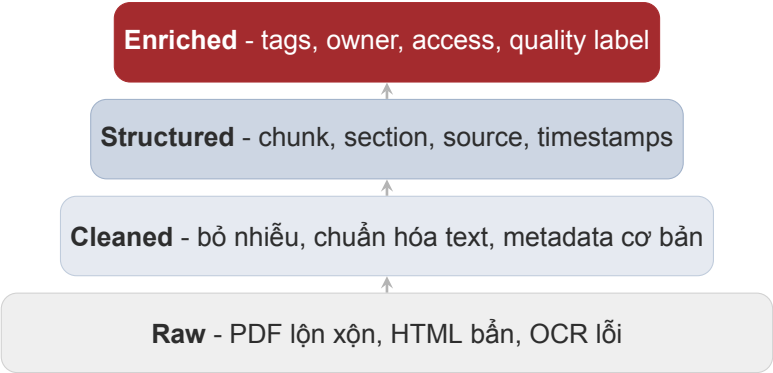
- Gắn liền với session / user hiện tại
- Thường ngắn gọn, inject trực tiếp vào prompt
- Có thể đến từ profile store hoặc session state
- Giúp agent cá nhân hóa câu trả lời

Ví dụ thực tế

- User profile: tên, ngôn ngữ, vai trò
- Lịch sử hội thoại gần nhất
- Giỏ hàng hiện tại, trang đang xem
- Kênh liên hệ (chat, email, phone)

Retrieval fit: Trung bình — inject trực tiếp hoặc lookup đơn giản, ít khi cần semantic search.

Data Quality Pyramid



Lưu ý: Sai lầm phổ biến: index ngay dữ liệu **raw** rồi kỳ vọng retrieval sẽ tự chữa mọi vấn đề.

Ví Dụ: Data Quality Ảnh Hưởng Retrieval Như Thế Nào?

VINUNIVERSITY

Trước khi clean

Query: “thời hạn đổi trả”

Top-1 retrieved:
“...trang 3... header lặp... \n\n mục lục... 30 ngày... xem thêm trang 7...”

Answer: Hallucinate vì context quá nhiễu, agent lấy thông tin sai.

Sau khi clean + chunk

Query: “thời hạn đổi trả”

Top-1 retrieved:
“Khách hàng có 30 ngày kể từ ngày nhận hàng để yêu cầu đổi trả. Sản phẩm phải còn nguyên seal.”

Answer: Chính xác, grounded, có trích nguồn.

> Cùng một model, cùng một query — chỉ khác data quality đã tạo ra kết quả hoàn toàn khác.

Data Ownership Và Governance

- ☑ **Ai sở hữu dữ liệu?** team nào chịu trách nhiệm cập nhật?
- ☑ **Ai được phép truy cập?** dữ liệu nào cần ACL, dữ liệu nào public nội bộ?
- ☑ **Data freshness** bao lâu phải re-index?
- ☑ **PII / sensitive fields** có cần mask trước khi embed không?
- ☐ **Không index mọi thứ theo kiểu "cứ nạp hết vào vector DB đã".**

PII Masking Trước Khi Embed

Loại PII	Ví dụ	Kỹ thuật mask	Mức rủi ro
Tên cá nhân	“Nguyễn Văn A”	Thay bằng `[PERSON]`	Trung bình
Số điện thoại	“0912-xxx-xxx”	Regex replace	Cao
Email	“user@email.com”	Hash hoặc remove	Cao
CMND / CCCD	“012345678901”	Xóa hoàn toàn	Rất cao
Địa chỉ	“123 Lê Lợi, Q.1”	Generalize “Q.1, HCM”	Trung bình

Mask trước khi embed — không bao giờ lưu raw PII trong vector store

02

Agent Memory Architecture

Memory không chỉ là lưu nhiều hơn; nó là quyết định xem agent được nhớ gì, quên gì, và truy gì khi cần

Short-term Memory vs Long-term Memory

VINUNIVERSITY

Short-term Memory

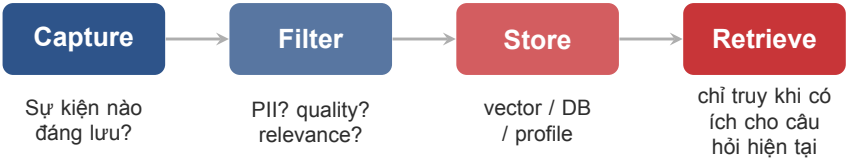
- Nằm trong context window
- Giữ lịch sử gần đây và task hiện tại
- Rẻ để dùng, nhưng dễ đầy token
- Phù hợp cho session logic ngắn

Long-term Memory

- Nằm ngoài context window
- Thường là vector store, DB, profile store
- Phải truy xuất khi cần
- Phù hợp cho tri thức tích lũy và user history chọn lọc

> Context window không phải là vector store. Agent chỉ “nhớ dài hạn” khi có cơ chế truy xuất hoặc lưu trữ bên ngoài.

Memory Lifecycle



Lưu ý: Memory tốt là memory có **chính sách**. Lưu tất cả thường làm tăng chi phí, nhiễu, và rủi ro quyền riêng tư.

Cái Gì Không Nên Gọi Là Memory?

- Một prompt dài hơn không tự biến thành memory.
- Một file PDF upload một lần không phải memory nếu agent không truy được lại có chủ đích.
- Toàn bộ chat history không phải lúc nào cũng đáng inject.
- “Lưu cho chắc” thường tạo ra context nhiễu hơn là context hữu ích.

> Memory là **data + policy + retrieval**. Thiếu một trong ba thứ đó thì hệ thống khó ổn định.

Context Window Budget — Phân Bổ Token

- **System prompt:** instructions, persona, guardrails — cố định mỗi request
- **Retrieved context:** top-k chunks inject vào — chiếm nhiều token nhất
- **Conversation history:** các turn gần nhất — cắt bớt khi dài
- **Generation budget:** token dành cho output — quá ít sẽ bị cắt giữa câu

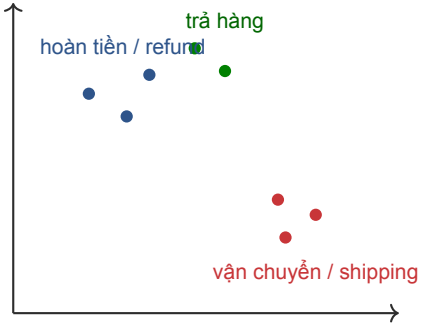
Lưu ý: Nếu retrieved context chiếm quá nhiều, agent sẽ không đủ token để trả lời tốt. Luôn **budget trước khi inject**.

03

Embeddings

Embedding biến ngôn ngữ thành không gian toán học để máy có thể so sánh nghĩa thay vì chỉ so chữ

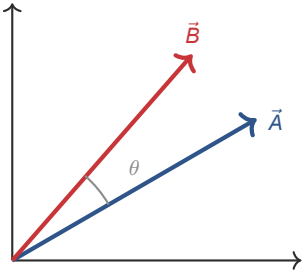
Embedding Là Gì?



Minh họa trực quan: câu có nghĩa gần nhau sẽ ở gần nhau hơn trong vector space.

- Text được biến thành vector nhiều chiều.
- Từ đó, máy có thể đo **độ gần về nghĩa**.
- Đây là nền tảng cho semantic search, clustering, dedup, recommendation.

Cosine Similarity — Đo Độ Gần Về Nghĩa



$$\text{cosine_sim}(\vec{A}, \vec{B}) = \frac{\vec{A} \cdot \vec{B}}{||\vec{A}|| \cdot ||\vec{B}||}$$

- **1.0** = hoàn toàn cùng hướng (cùng nghĩa)
- **0.0** = vuông góc (không liên quan)
- **–1.0** = ngược hướng (nghĩa đối lập)
- Không phụ thuộc vào độ dài vector

> Hầu hết vector DB dùng cosine similarity làm metric mặc định cho semantic search.

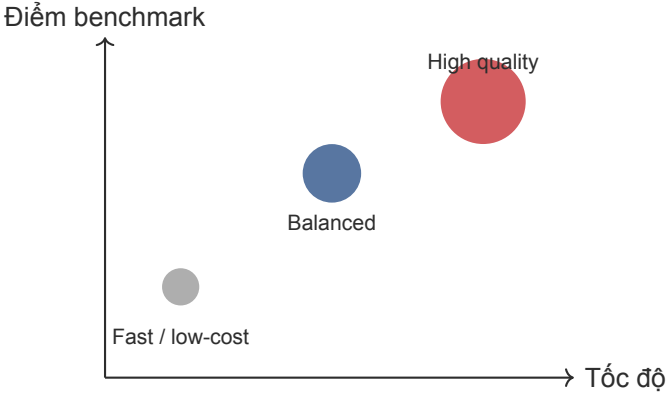
Embedding Dùng Để Làm Gì Trong AI Product?

- **Semantic search**: tìm đoạn liên quan dù không trùng keyword
- **Clustering**: gom nội dung gần nghĩa
- **Dedup**: phát hiện đoạn tương tự nhau
- **Recommendation**: tìm item gần hành vi / intent

Sai lầm phổ biến

Embedding không tự tạo ra "sự thật". Nếu text nguồn sai, thiếu, hoặc chunk xấu thì retrieval vẫn tệ.

Benchmark Framing: Quality, Speed, Size



Illustrative redraw based on the MTEB speed benchmark: quality, throughput, and embedding size are trade-offs.

- Không có model embedding "tốt nhất" cho mọi use case.
- Chọn model theo **quality**, **latency**, **storage**, **language coverage**.
- Với product thực tế, một model cân bằng thường tốt hơn model mạnh nhất nhưng chậm và đắt.

Các Embedding Model Phổ Biến (2025–2026)

Model	Dims	Max tokens	Provider	Ghi chú
text-embedding-3-small	1536	8191	OpenAI	Rẻ, nhanh, baseline tốt
text-embedding-3-large	3072	8191	OpenAI	Quality cao hơn
voyage-3	1024	32000	Voyage AI	Code & multilingual mạnh
Cohere embed-v4	1024	512	Cohere	Multilingual, search-tuned
bge-m3	1024	8192	BAAI (open)	Open-source, MTEB top tier

Chọn model theo use case, không chỉ theo MTEB score

Embeddings — Python Example

```python
from openai import OpenAI

client = OpenAI()

resp = client.embeddings.create(
    model="text-embedding-3-small",
    input=[
        "Chinh sach hoan tien",   # Chính sách hoàn tiền
        "Quy dinh doi tra"        # Quy định đổi trả
    ]
)

print(len(resp.data[0].embedding))
```

Embedding Dưới Hood — Không Chỉ Là API Call

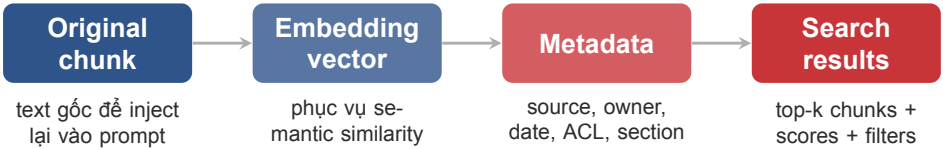
- **Tokenizer:** chia text thành subword units (BPE / SentencePiece)
- **Transformer layers:** self-attention giúp mỗi token "nhìn" toàn bộ context
- **Pooling:** gom tất cả token representations thành 1 vector duy nhất
- **Training:** contrastive learning — câu giống nhau gần, câu khác nhau xa

04

Vector Store

Vector store không chỉ lưu vector; nó lưu cả nguồn, metadata, và khả năng lọc để retrieval có ngữ cảnh đúng

Vector Store Lưu Những Gì?



Metadata Quan Trọng Không Kém Similarity

Trường metadata	Ví dụ	Tác dụng
Source / file name	`refund-policy-v3.pdf`	truy vết và hiển thị nguồn
Category	`finance`, `support`, `legal`	lọc đúng domain trước khi search
Time / freshness	`2026-03-01`	tránh dùng nội dung cũ
Access level	`public-internal`, `restricted`	giới hạn truy cập
Section / chunk id	`returns.section.4`	debug và cite chính xác hơn

Retrieval tốt cần semantic similarity kết hợp metadata filtering

Top-k, Score Threshold, Filter

- **Top-k**: lấy quá ít thì thiếu context, lấy quá nhiều thì nhiễu.
- **Score threshold**: giúp bỏ các chunk liên quan yếu.
- **Attribute filter**: lọc theo region, team, thời gian, loại tài liệu.
- **Hybrid thinking**: semantic similarity tốt, nhưng nhiều bài toán vẫn cần keyword / metadata song song.

> Retrieval tốt hiếm khi đến từ “đổi model” trước tiên; thường đến từ **chunk đúng + metadata đủ + filter hợp lý**.

Vector Database Landscape

DB	Hosting	Language	Filtering	Scale	Best for
Chroma	Local	Python	Basic	Small	Prototype, học tập
Pinecone	Cloud	Any	Tốt	Large	Production SaaS
Weaviate	Hybrid	Any	Tốt	Large	Hybrid search
Qdrant	Hybrid	Rust	Rất tốt	Large	Filtering phức tạp
pgvector	DB addon	SQL	SQL where	Medium	Team đã dùng Postgres
FAISS	Local lib	C++/Py	Không	Any	Research, bench-mark

Chọn DB theo giai đoạn sản phẩm và yêu cầu vận hành

Chroma — Local Vector Store Example

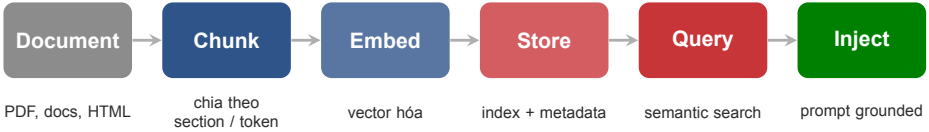
```python
import chromadb

client = chromadb.Client()
collection = client.create_collection("policies")

collection.add(
    ids=["p1"],
    documents=["Khach hang co 30 ngay de doi tra san pham."],
    # «Khách hàng có 30 ngày để đổi trả sản phẩm.»
    metadatas=[{"category": "support", "lang": "vi"}],
)
```

05

Kết Nối Agent Với Data

Retrieval pipeline là chiếc cầu nối giữa dữ liệu riêng và hành vi của agent

Retrieval Pipeline Chuẩn



Pipeline redraw

based on OpenAI Retrieval / File Search guides.

Chunking: Quá To Hay Quá Nhỏ Đều Trả Giá

VINUNIVERSITY

Chunk quá to

Dính nhiều chủ đề vào cùng một vector, retrieve trúng nhưng inject rất nhiễu.

Chunk hợp lý

Một ý / một section tương đối trọn vẹn, có overlap vừa đủ để giữ nghĩa.

Chunk quá nhỏ

Mất ngữ cảnh, retrieve nhiều mảnh rời rạc, khó tổng hợp thành câu trả lời tốt.

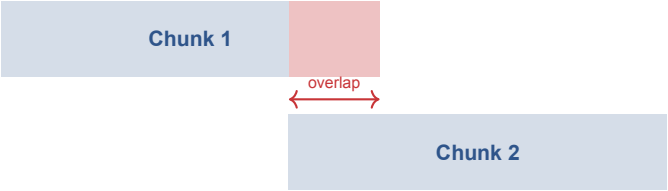
> Bắt đầu đơn giản với chunk theo section / heading, rồi tối ưu sau bằng eval thay vì đoán cảm tính.

Các Phương Pháp Chunking Phổ Biến

Phương pháp	Cách hoạt động	Ưu điểm	Nhược điểm
Fixed-size token	Cắt mỗi N tokens	Đơn giản, dễ implement	Cắt giữa câu / ý
Sentence-based	Tách theo câu hoàn chỉnh	Giữ nghĩa câu	Câu dài / ngắn không đều
Section / heading	Tách theo heading (H1, H2)	Chunk trọn vẹn về ý	Cần tài liệu có structure
Recursive	Thử separator lớn → nhỏ	Linh hoạt	Phức tạp hơn
Semantic	Gom câu cùng chủ đề	Chất lượng cao nhất	Chậm, cần embedding

Không có "best" strategy — chọn theo đặc điểm tài liệu

Chunk Overlap — Tại Sao Cần?



Không overlap: mất nghĩa ở biên chunk

Có overlap: giữ liên tục ngữ cảnh

- **Overlap thường 10–20%** kích thước chunk (e.g., 50–100 tokens cho chunk 512 tokens)
- Quá nhiều overlap → duplicate context, tốn storage, nhiễu kết quả
- Quá ít overlap → mất nghĩa ở ranh giới giữa hai chunk

Retrieval Khác Gì Memory?

VINUNIVERSITY

Retrieval

- tìm context liên quan cho câu hỏi hiện tại
- thường đọc từ knowledge base
- trọng tâm là relevance và grounding

Memory

- lưu trạng thái, sở thích, lịch sử chọn lọc
- có thể từ user profile hoặc interaction history
- trọng tâm là continuity và personalization

Lưu ý: Nhiều hệ thống dùng cả hai: **retrieval** để biết domain facts, **memory** để biết user này là ai và đã làm gì trước đó.

Retrieve Rồi Inject — Python Example

```
query = "Chinh sach doi tra ap dung trong bao lau?"
# = "Chính sách đổi trả áp dụng trong bao lâu?"
results = collection.query(query_texts=[query], n_results=3)

context = "\n".join(results["documents"][0])

prompt = f"""Dua tren nguon sau, hay tra loi ngan gon.
# «Dựa trên nguồn sau, hãy trả lời ngắn gọn.»

Nguon:
{context}

Cau hoi: {query}
"""
```

So Sánh: LLM Chay vs RAG-Enhanced

VINUNIVERSITY

LLM Chay (No Retrieval)

- Chỉ dựa vào training data
- Dễ hallucinate với domain-specific
- Không cite được nguồn cụ thể
- Không cập nhật được data mới

RAG-Enhanced

- Grounded trên dữ liệu thật
- Giảm hallucination đáng kể
- Trích nguồn cho user verify
- Cập nhật bằng re-index, không retrain

> RAG thêm latency (retrieve step) và complexity (pipeline), nhưng đổi lại **accuracy** và **trustworthiness** cao hơn nhiều.

Hosted Retrieval vs Self-managed Retrieval

VINUNIVERSITY

Hosted / managed

File Search / managed retrieval giúp đi nhanh, giảm code hạ tầng, phù hợp demo và PoC.

Self-managed

Dùng Chroma / Faiss / vector DB khi cần kiểm soát chunking, metadata, pipeline, hoặc cost path chi tiết hơn.

> Ngày 7 chỉ cần hiểu **trade-off**. Chưa cần sa sâu vào vector DB war hay tối ưu indexing nâng cao.

Common Failure Patterns Trong Retrieval

Pattern	Biểu hiện	Cách khắc phục
Chunk quá to	Retrieve trúng nhưng answer nhiễu	Chunk nhỏ hơn, theo section
Thiếu metadata	Không filter được, lẫn domain	Thêm category, source, date
Top-k quá cao	Inject quá nhiều context	Giảm k, thêm score threshold
Data cũ / stale	Trả lời đúng... nhưng outdated	Re-index định kỳ, version tracking
Query mơ hồ	Retrieve sai hoàn toàn	Query rewriting / expansion

Debug retrieval bắt đầu từ nhận diện pattern lỗi

Đo Lường Retrieval Quality

VINUNIVERSITY

Metrics quan trọng

- **Precision@k**: trong k kết quả, bao nhiêu thực sự relevant?
- **Recall@k**: trong tất cả docs relevant, bao nhiêu nằm trong top-k?
- **MRR** (Mean Reciprocal Rank): kết quả relevant đầu tiên ở vị trí nào?
- **NDCG**: thứ tự rank có tốt không?

Cách eval đơn giản cho lab

- Viết 10–20 test queries với expected docs
- Chạy retrieval, so sánh top-k vs expected
- Tính Precision@3 và Recall@3
- Ghi nhận failure cases: query nào retrieve sai?

> Eval retrieval **trước khi** eval generation — nếu retrieve sai thì generate không thể đúng.

06

Hands-on 7

Mục tiêu của lab là nối được dữ liệu riêng vào hệ thống AI theo pipeline tối thiểu nhưng đúng bản chất

Lab 7: Vector Store Integration — Overview

1. Chọn một bộ tài liệu nhỏ: FAQ, SOP, policy, internal docs
2. Chuẩn hóa text và quyết định metadata tối thiểu
3. Chunk dữ liệu theo section
4. Embed và lưu vào vector store
5. Viết semantic search demo
6. Inject top-k results vào answer function hoặc mini agent

> Một knowledge base nhỏ nhưng truy đúng và trả lời grounded tốt có giá trị học tập hơn một data set lớn nhưng nhiễu.

Lab Step 1: Chọn & Chuẩn Bị Data

VINUNIVERSITY

Yêu cầu tối thiểu

- 5–15 tài liệu (FAQ, SOP, policy)
- Mỗi tài liệu 1–5 trang
- Text đã extract sạch (không còn header/footer lặp)
- Ghi lại nguồn gốc mỗi file

Metadata tối thiểu cần ghi

- `source`: tên file gốc
- `category`: domain / loại tài liệu
- `date`: ngày tạo hoặc cập nhật
- `language`: `vi` / `en`

Lab Step 2: Chunking Code Example

```python
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,
    chunk_overlap=64,
    separators=["\n\n", "\n", ". ", " "]
)

docs = load_documents("./data/")  # your loader
chunks = []
for doc in docs:
    parts = splitter.split_text(doc["text"])
    for i, part in enumerate(parts):
        chunks.append({
            "id": f"{doc['source']}_chunk_{i}",
            "text": part,
            "metadata": {"source": doc["source"],
                         "category": doc["category"]}
        })
```

Lab Step 3: Embed & Store

```python
import chromadb
from openai import OpenAI

client_ai = OpenAI()
client_db = chromadb.Client()
collection = client_db.create_collection("lab7_kb")

for chunk in chunks:
    resp = client_ai.embeddings.create(
        model="text-embedding-3-small",
        input=[chunk["text"]]
    )
    collection.add(
        ids=[chunk["id"]],
        embeddings=[resp.data[0].embedding],
        documents=[chunk["text"]],
        metadatas=[chunk["metadata"]]
    )
```

Lab Step 4: Semantic Search Demo

```
query = "thoi han doi tra san pham"
# = "thời hạn đổi trả sản phẩm"
results = collection.query(
    query_texts=[query],
    n_results=3,
    where={"category": "support"}  # metadata filter
)

for i, doc in enumerate(results["documents"][0]):
    score = results["distances"][0][i]
    print(f"[{i+1}] score={score:.3f}")
    print(f"    {doc[:120]}...")
```

Lab Step 5: Answer Function Có Retrieved Context

```
def answer_with_context(query, collection, model="gpt-4o-mini"):
    results = collection.query(query_texts=[query], n_results=3)
    context = "\n---\n".join(results["documents"][0])

    response = client_ai.chat.completions.create(
        model=model,
        messages=[{
            "role": "user",
            "content": f"""Dua tren nguon sau, tra loi ngan gon.
Neu khong tim thay, noi 'Khong co thong tin'.
# «Dựa trên nguồn sau, trả lời ngắn gọn. Nếu không tìm thấy, nói 'Không có thông tin'.»

Nguon:\n{context}\n\nCau hoi: {query}"""
        }]
    )
    return response.choices[0].message.content
```

Deliverable Và Assessment

VINUNIVERSITY

Deliverable

- Data inventory 1 trang
- Script chunk + embed + index
- Query demo với 3 câu hỏi test
- Mini answer function có dùng retrieved context

Instructor check

- Query có retrieve đúng tài liệu không?
- Metadata có giúp lọc tốt hơn không?
- Câu trả lời có grounded hơn baseline không?
- Có biết chỉ ra failure case không?

Checklist Trước Khi Chốt Lab

- ☑ Dữ liệu có owner và source rõ ràng
- ☑ Chunking có lý do, không cắt bừa theo số ký tự
- ☑ Metadata có ít nhất 2 trường hữu ích
- ☑ Có test query semantic, không chỉ test exact keyword
- ☐ Không chấp nhận câu trả lời nghe hay nhưng không chỉ ra được nguồn

Data Inventory — Mẫu Template

Tài liệu	Loại	Owner	Freshness	PII?	Retrieval fit
FAQ v3.pdf	Knowledge	Support	Monthly	Không	Cao
Order status API	Operational	Eng	Real-time	Có	Thấp (API)
User profile	Contextual	Product	Per-session	Có	Inject trực tiếp
Legal contracts	Knowledge	Legal	Quarterly	Có	Cao (cần mask)

Template gợi ý — điều chỉnh theo domain thực tế

Anti-patterns: Những Gì Không Nên Làm

1. **“Nạp hết vào vector DB rồi tính sau”** — Garbage in, garbage out. Clean trước, index sau.
2. **Chunk cố định 500 chars cho mọi loại tài liệu** — Mỗi loại doc cần strategy khác nhau.
3. **Không có metadata** — Không filter được → retrieve lẫn domain → answer sai.
4. **Không eval retrieval** — Chỉ eval generation mà bỏ qua retrieval → debug mù.
5. **Embed raw PII** — Vi phạm privacy, rủi ro pháp lý, khó thu hồi.
6. **Top-k = 10 “cho chắc”** — Inject quá nhiều context → nhiễu, tốn token, chậm.

Best Practices: Tổng Hợp

- ☑ **Clean data trước khi embed** — bỏ header/footer lặp, chuẩn hóa encoding
- ☑ **Chunk theo structure** (heading, section) thay vì fixed-size
- ☑ **Thêm metadata** ít nhất: source, category, date
- ☑ **Overlap 10–20%** giữa các chunk liền kề
- ☑ **Eval retrieval riêng** trước khi eval end-to-end
- ☑ **Start small**: 10–20 docs, 3–5 test queries, iterate
- ☐ **Không fine-tune embedding model** trước khi đã tối ưu hết data + chunking + metadata

Distance Metrics: Cosine vs Euclidean vs Dot Product

VINUNIVERSITY

Cosine

Đo góc giữa 2 vector.

Không phụ thuộc độ dài.

Mặc định cho hầu hết embedding models.

Euclidean (L2)

Khoảng cách "đường chim bay".

Phụ thuộc magnitude.

Phù hợp khi vectors đã normalize.

Dot Product

Nhanh nhất.

Kết hợp direction + magnitude.

Tốt khi cần ưu tiên "confidence".

Với OpenAI embeddings (đã normalized), cosine $\approx$ dot product. Bắt đầu với **cosine**, chỉ đổi khi có lý do cụ thể.

Indexing Algorithms — Approximate Nearest Neighbor

Vấn đề: Brute-force search O(n) quá chậm khi có hàng triệu vectors.
Giải pháp: ANN algorithms đánh đổi *một chút accuracy* để được *tốc độ gấp 100–1000x*.

Các thuật toán phổ biến:

- **HNSW** (Hierarchical Navigable Small World) — mặc định ở hầu hết DB
- **IVF** (Inverted File Index) — phân cụm rồi search trong cluster
- **PQ** (Product Quantization) — nén vector để tiết kiệm RAM

> Ở quy mô lab (<10K vectors), brute-force vẫn đủ nhanh. ANN chỉ cần khi scale lên production.

> **Lưu ý:** Đây là kiến thức nền. Không cần implement ANN ở Day 7 — chỉ cần biết nó tồn tại.

Bonus: Multi-modal Embeddings

VINUNIVERSITY

Không chỉ text

- **Image embeddings**: CLIP, SigLIP
- **Audio embeddings**: CLAP, Whisper
- **Code embeddings**: Voyage Code, CodeBERT
- **Cross-modal**: tìm ảnh bằng text query và ngược lại

Ứng dụng thực tế

- E-commerce: search sản phẩm bằng ảnh
- Medical: tìm X-ray tương tự
- Code: semantic code search
- Creative: text-to-image retrieval

Day 7 focus vào text embedding. Multi-modal là hướng mở rộng sau.

Khi Nào KHÔNG Cần RAG?

VINUNIVERSITY

Không nên dùng RAG khi:

- Câu hỏi hoàn toàn nằm trong training data (general knowledge)
- Data đã fit trong context window (đủ nhỏ để inject hết)
- Cần reasoning / tính toán, không cần factual lookup
- Task là creative writing, brainstorming

Nên dùng RAG khi:

- Cần domain-specific, proprietary knowledge
- Data thay đổi thường xuyên
- Cần cite nguồn, trích dẫn cụ thể
- Knowledge base quá lớn để fit trong context
- Cần kiểm soát access (ACL per doc)

Lưu ý: RAG không phải silver bullet. Nếu bài toán không cần external knowledge, RAG chỉ thêm latency và complexity.

Ví Dụ End-to-End: Customer Support Bot

Knowledge data: FAQ đổi trả, chính sách bảo hành, hướng dẫn sử dụng

Operational data: trạng thái đơn hàng (qua API)

Contextual data: lịch sử chat, user tier (VIP/regular)

Chunking: theo section heading

Metadata: category, language, version

Eval: 20 test queries, Precision@3 $>$ 0.8

Fallback: "Tôi không tìm thấy thông tin, xin chuyển cho nhân viên."

Quay Lại Câu Hỏi Đầu Bài

"Agent trả lời sai vì model yếu, hay vì nó không có đúng dữ liệu để suy luận?"

- Hầu hết trường hợp: **vì data**, không phải vì model.
- Data bẩn + chunk sai + không metadata → model mạnh nhất cũng hallucinate.
- Pipeline tốt (clean → chunk → embed → store → retrieve → inject) giải quyết 80% vấn đề.

> Đầu tư vào **data quality** và **retrieval pipeline** trước khi nghĩ đến đổi model.

Key Takeaways

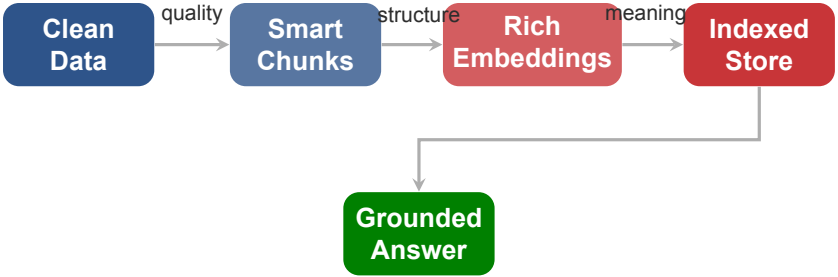
07

Day 07 đặt nền cho RAG và mọi hệ thống AI dùng tri thức riêng

4 Ý Cần Nhớ Sau Buổi Học

1. **Data quality** thường quan trọng hơn đổi sang model đắt hơn.
2. **Embedding** là lớp dịch ngôn ngữ sang không gian có thể so sánh nghĩa.
3. **Vector store** là bộ nhớ dài hạn có thể tìm kiếm bằng ngữ nghĩa và metadata.
4. **Retrieval pipeline** là cầu nối từ dữ liệu riêng tới câu trả lời grounded của agent.

Mental Model: Từ Data Đến Grounded Answer



Lưu ý: Mỗi bước yếu sẽ kéo toàn bộ pipeline xuống. Hãy đầu tư đều vào từng bước thay vì chỉ tối ưu phần cuối.

Tài Liệu Tham Khảo

1. Stanford HAI. *The 2025 AI Index Report*. `https://hai.stanford.edu/ai-index/2025-ai-index-report`
2. OpenAI Docs. *Embeddings Guide*. `https://platform.openai.com/docs/guides/embeddings`
3. OpenAI Docs. *Retrieval Guide*. `https://platform.openai.com/docs/guides/retrieval`
4. OpenAI Docs. *File Search Guide*. `https://platform.openai.com/docs/guides/tools-file-search`
5. Hugging Face. *MTEB: Massive Text Embedding Benchmark*. `https://huggingface.co/blog/mteb`
6. Chroma Documentation. *Getting Started / Collections / Query*. `https://docs.trychroma.com/`
7. Karpukhin et al. *Dense Passage Retrieval for Open-Domain QA*. arXiv:2004.04906
8. Malkov & Yashunin. *Efficient and Robust ANN using HNSW Graphs*. arXiv:1603.09320

Tiếp Theo & Bài Tập

VINUNIVERSITY

Bài tập sau buổi

- Rà lại knowledge base của nhóm và bỏ 20% nội dung nhiễu nhất
- Viết 5 test query để đo retrieval quality
- Chỉ ra 2 failure cases do chunking hoặc metadata
- Thử thay đổi chunk_size và overlap, ghi nhận sự khác biệt

Preview Day 08

Ngày 8 đi tiếp sang **RAG pipeline**: indexing, retrieval, generation, evaluation.

HÃY SUY NGHĨ...

“Cảm Ơn! — Q&A”

Giữ câu hỏi này trong đầu khi học bài hôm nay